

Enunciado do Trabalho Prático

1 Introdução

Com este trabalho prático pretende-se que os estudantes de *Processamento de Linguagens* adquiram experiência na conceção e implementação de analisadores léxicos e sintáticos, bem como na definição de ações semânticas que traduzem a linguagem de entrada.

O problema proposto é a implementação de uma linguagem simplificada de programação funcional. Em particular, a solução passa pelo desenvolvimento de uma aplicação para o processamento da linguagem LFun (*Linguagem Funcional*) descrita na secção 2.1,

O processo para a realização deste trabalho prático deverá passar pelas seguintes fases:

1. especificar a gramática concreta da linguagem de entrada;
2. desenvolver um reconhecedor léxico (recorrendo à biblioteca *lex*) para processar os símbolos terminais identificados na gramática concreta, e testar esse reconhecedor com um conjunto representativo de *palavras* da linguagem de entrada;
3. desenvolver um reconhecedor sintático (recorrendo à biblioteca *yacc*) para reconhecer a gramática concreta, e testar esse reconhecedor com alguns exemplos frases da linguagem de entrada. e testar esse reconhecedor com um conjunto representativo de *frases* da linguagem de entrada;
4. definir uma *árvore de sintaxe abstrata* para representar a linguagem de entrada, e associar ações semânticas, de tradução, às regras de produção da gramática de forma a determinar a respetiva árvore de sintaxe abstrata;
5. implementar análise semântica com tabela de símbolos e verificação de tipos;
6. execução, interpretação ou geração de código que produza a resposta solicitada, através da avaliação da árvore de sintaxe abstrata.

Na secção seguinte é apresentado o enunciado do tema proposto para este trabalho prático. Na secção 3 são apresentadas as regras para o desenvolvimento deste trabalho prático.

2 Enunciado

Neste trabalho prático pretende-se que os estudantes implementem uma aplicação na linguagem de programação **Python**, usando a biblioteca **ply**, que interprete uma linguagem capaz de especificar algumas instruções que habitualmente encontramos em linguagens de programação.

A aplicação a desenvolver deverá começar por ler um ficheiro de texto (com extensão **.lf**) contendo uma sequência de comandos de especificação da linguagem, aplique esses comandos de forma a calcular o resultado pretendido. O resultado de processar o ficheiro de texto **entrada.lf** é apresentado ao utilizador

no terminal (opcionalmente, o programa poderá gerar um ficheiro com a respetiva implementação em código C). Caso não seja apresentado o ficheiro de entrada, os comandos devem ser lidos do terminal à medida que o utilizador os vai inserindo.

2.1 Descrição da Linguagem

A seguinte descrição e sequência de fragmentos ilustra, sem ser normativa, o estilo geral esperado para programas na linguagem LFun:

A. expressões

As expressões correspondem a uma construção sintática que, quando avaliada, produz um valor. No desenvolvimento da linguagem, as expressões devem ser introduzidas de forma progressiva, começando por formas simples e evoluindo para construções mais ricas, como condicionais, avaliação de casos e aplicação de funções.

Considere-se que a linguagem suporta *expressões aritméticas* e *expressões booleanas* (condições), definindo de forma explícita a sintaxe adotada, a precedência dos operadores e as respetivas regras dos tipos de dados.

As expressões aritméticas são expressões cujo resultado é um valor do tipo `Int`. Em geral, podem envolver: constantes inteiras; identificadores associados a valores inteiros; operadores aritméticos; uso de parênteses para explicitar a ordem de avaliação.

```
3 + 4;  
resultado: 7 tipo: Int  
10 - 2 * 3;  
resultado: 4 tipo: Int  
(8 + 2) * 5;  
resultado: 50 tipo: Int
```

As expressões booleanas são expressões cujo resultado é um valor do tipo `Bool`. Em geral, resultam de comparações, negações ou combinações lógicas entre condições.

Exemplos:

```
3 < 5;  
resultado: true tipo: Bool  
10 == 7 ;  
resultado: false tipo: Bool  
(4 + 1) == 5 ;  
resultado: true tipo: Bool
```

A implementação deverá assegurar que as expressões são semanticamente coerentes com os respetivos tipos. Em particular, espera-se que o analisador semântico rejeite situações como: utilização de valores booleanos em operações estritamente aritméticas; aplicação de operadores aritméticos a valores não inteiros; comparação entre valores de tipos incompatíveis, salvo se o grupo definir e justificar explicitamente esse comportamento.

A definição exata do conjunto de operadores suportados, da sintaxe concreta das expressões booleanas e das regras de avaliação associadas constitui uma decisão do grupo, a qual deverá ser documentada e justificada no relatório.

B. definições

As definições são introduzidas com `let` e devem ter indicado o respetivo tipo, associando um identificador a uma expressão (cujo valor deverá respeitar o tipo indicado). O identificador deve começar obrigatoriamente por uma letra, e nas posições seguintes pode ter qualquer combinação de letras, dígitos e *underscore* `_`.

```
let base : Int = 10;
-- base: tem o tipo Int, e o resultado esperado da avaliação: 10

let ativo : Bool = true;
-- ativo: tem o tipo Bool, e o resultado esperado da avaliação: true

let area : Int = 4 * 5;
-- area tem o tipo Int, e o resultado esperado da avaliação: 20
```

Nestes casos, `base`, `ativo` e `area` são identificadores definidos uma única vez, ficando associados aos valores produzidos pelas respetivas expressões. Assume-se que neste tipo de definições, não se trata de uma variável mutável que possa ser atualizada ao longo do programa, por isso não são aceites expressões como `x=10`; ou `i=i+1`;

De qualquer forma o valor poderá ser usar nas expressões seguintes:

```
let largura : Int = 8;
let altura : Int = 5;
let area : Int = largura * altura;
-- area tem o tipo Int, e o resultado esperado da avaliação: 40

let teste : Bool = altura < largura;
-- teste tem o tipo Bool, e o resultado esperado da avaliação: false

{- os seguinte exemplo deve ser rejeitado pelo analisador semântico,
   porque a expressão definida tem tipo `Bool`, mas o identificador
   foi declarado com tipo `Int`. -}
let valor : Int = true;
```

C. funções

As funções devem ter uma assinatura explícita com `fun`, que define o tipo dos argumentos e o tipo do valor devolvido, permitindo ao analisador semântico verificar a coerência da definição correspondente. A definição operacional de uma função é também introduzida com `let`, após a respetiva assinatura.

```
fun inc : Int -> Int;
let inc x = x + 1;
```

Neste exemplo, a primeira linha declara a assinatura da função `inc`, e a segunda linha apresenta a sua definição. O resultado esperado de `inc(5)` é 6 (do tipo `Int`).

```
fun dobro : Int -> Int;
let dobro n = n * 2;
```

```
dobro(7); -- dobro(7)= 7*2 = 14
```

```
fun ePositivo : Int -> Bool;  
let ePositivo n = n > 0;
```

```
ePositivo(3); -- true  
ePositivo(-2); -- false
```

D. condicional e seleção de casos

Além de expressões aritméticas, a linguagem tem mecanismos que permitem selecionar resultados diferentes em função de condições distintas. Temos duas construções complementares: a estrutura condicional **if**, adequada para decisões binárias; e a seleção de casos **when**, adequada para organizar de forma mais clara situações com vários casos possíveis.

Estas duas construções permitem exprimir decisões, com o objetivo de calcular um resultado a partir da avaliação de condições ou padrões.

A estrutura **if** é a forma mais simples de seleção. Permite escolher entre duas expressões alternativas em função de uma condição booleana. Vamos considerar a seguinte forma geral:

if condição **then** **exp1** **else** **exp2** A condição deve ter tipo **Bool**, e se a condição for verdadeira, o resultado é o valor de **exp1**; se a condição for falsa, o resultado é o valor de **exp2**; Ambos os ramos devem produzir valores compatíveis entre si.

```
fun absoluto : Int -> Int;  
let absoluto n = if n < 0 then 0 - n else n;  
  
absoluto(-5); -- resultado esperado de absoluto(-5)=0-(-5)= 5  
absoluto(8);  -- resultado esperado de absoluto(8) = 8
```

— A linguagem suporta uma forma de avaliação de casos por correspondência de padrões **when**. Esta construção permite analisar um valor e escolher o resultado correspondente ao primeiro padrão aplicável, permitindo organizar explicitamente diferentes casos, cada um associado a um padrão.

```
fun eZero : Int -> Bool;  
let eZero n =  
  when (n) is  
    0 -> true;  
    _ -> false;  
end;  
  
fun avalia : Int -> Bool;  
let avalia x =  
  when ( x ) is  
    -1,  
    1  -> true  ;    --  lista de padrões alternativos: -1, 1  
    0  -> false ;  
    _  -> true  ;    --  _ representa o otherwise  
end;
```

```

avalia(-1);  -- true
avalia(0);   -- false
avalia(-1);  -- true
avalia(2);   -- true

fun boolToInt : Bool -> Int;
let boolToInt b =
  when b is
    true  -> 1;
    false -> 0;
  end;

boolToInt(true);  -- 1
boolToInt(false); -- 0

```

E. comentários

Os comentários apenas de uma linha são precedidos por `--` (todo o texto seguinte é ignorado). No caso dos comentários em mais do que uma linha, temos os seguintes operadores `{- ... -}`.

F. outras instruções

Para efeitos de enriquecimento da linguagem desenvolvida, cada grupo poderá propor e implementar duas das seguintes extensões à linguagem base.

tipos `Char` e `String`, incluindo discussão sobre a sua representação e tratamento semântico;

pares (tuplos binários), com tipos declarados na forma `(T1, T2)`;

listas, com tipos declarados na forma `[T]`, de comprimento variável, incluindo o caso da lista vazia. A lista vazia é representada por `[]`, enquanto `h:tail` representa a lista obtida inserindo `h` na lista `tail`. Por exemplo, `1:[2,3]` representa a lista `[1,2,3]`;

suporte a funções recursivas

funções de ordem superior

Cada grupo deverá propor e justificar a respetiva solução, explicitando pelo menos: a sintaxe concreta adotada; as regras semânticas e de tipagem consideradas; as alterações introduzidas no analisador e na AST; o impacto da extensão na geração de código, quando aplicável; exemplos ilustrativos que demonstrem o funcionamento da funcionalidade proposta.

2.2 Observações

A avaliação deste trabalho terá em conta:

- a qualidade do reconhecedor léxico e da gramática implementados;
- a diversidade de operadores da linguagem apresentada no enunciado;
- a organização e qualidade do código desenvolvido;
- a estrutura da Árvore Abstrata de Sintaxe;

- a representação intermédia e geração de código. A solução deve ser pensada como genérica, com capacidade de adaptação, e não apenas para os exemplos descritos no enunciado.

3 Regras

- O trabalho tem carácter obrigatório para aprovação à unidade curricular, e deve ser realizado em grupo de, no máximo, *três* elementos;
- Durante as aulas dedicadas aos trabalhos cada grupo que deverá apresentar o trabalho desenvolvido até esse momento.
- O trabalho contempla uma apresentação e defesa individual em horário a agendar pelo docente. Esta defesa tem aprovação obrigatória, sendo que a falta à defesa corresponderá à não entrega do trabalho pelo estudante (i.e. avaliação de *zero* valores); Será agendado um horário com cada grupo para a apresentação do trabalho.
- O enunciado apresenta uma sequência de exercícios pensada para apoiar, de forma progressiva, o desenvolvimento do trabalho. Recomenda-se que os estudantes se envolvam ativamente na sua resolução e apliquem o esforço necessário em cada etapa, de modo a consolidarem as aprendizagens e a prepararem-se adequadamente para a defesa individual do trabalho entregue.
- O relatório deve introduzir o problema a ser resolvido, apresentar a abordagem seguida na sua resolução, quais os objectivos atingidos e quais os problemas encontrados. No relatório devem ser salientados todos os pontos que os alunos achem que poderão valorizar o seu trabalho em relação aos requisitos como, por exemplo, funcionalidades adicionais. Também deverá conter uma secção que descreva de que forma é que a aplicação foi testada. **O relatório poderá ter, no máximo, 10 páginas** (incluindo capa, índices, etc).

Devem ser colocadas todas as referências de *todas as fontes* consultadas durante a elaboração do trabalho (bibliográficas ou mesmo consultas *online*);

- A *data de entrega final* é aquela que foi estabelecida no início do semestre.
- Não serão aceites entregas ou melhorias após a data definida neste enunciado. Não serão aceites entregas ou melhorias nas épocas de exame (este trabalho apenas é válido para a avaliação da época em que é lançado).
- Quando o enunciado não for explícito em relação a um requisito específico, os alunos podem optar pela solução que parecer mais adequada, fazendo a sua apresentação e justificação no relatório. Dúvidas adicionais devem ser colocadas ao docente pessoalmente, de preferência nas aulas dedicadas ao desenvolvimento do trabalho.
- O esclarecimento de dúvidas acerca deste documento pode originar a publicação de novas versões.